00 Overview
01 Threads in DeepStream
02 The bus mental model
03 nvmultiurisrcbin & new nvstreammux
04 Scaling to 50 streams
05 RTSP failure & reconnect
06 Keeping it alive
07 Monitoring & status
08 Graceful reconnection
09 Detector, LPR, tracking, ReID
10 Production checklist

From a 4-stream demo to a 50-camera production pipeline

Everything from our discussion, written up: how the threads really work, the message bus, what `nvmultiurisrcbin` and the new `nvstreammux` do for you, and how to grow this into a pipeline that survives flaky RTSP cameras, server hiccups, and runs unattended for weeks — plus how to bolt on detection, license-plate recognition, tracking, and ReID.

GStreamer / PyGObject · new nvstreammux · REST add/remove · RTSP resilience · TAO inference

// 00

Overview: what the base pipeline is

The starting pipeline is a long-running REST server built around one composite element. Frames flow left to right; the server lets you add and remove cameras at runtime without restarting anything.

```
nvmultiurisrcbin → nvmultistreamtiler → nvvideoconvert → nvdsosd → fakesink
│
└─ bundles: per-source decode (nvurisrcbin ×N) + new nvstreammux + REST server
```

The key idea: `nvmultiurisrcbin` hides the per-source decoding, the batching, and an HTTP control server inside a single bin that exposes one static source pad. You POST JSON to `/api/v1/stream/add` and `/api/v1/stream/remove` to manage cameras live. The rest of this guide is about understanding *why* that works and how to make it withstand the real world.

> MENTAL ANCHOR
>
> Batching exists for **processing throughput** (run inference over N frames in one GPU pass). Tiling or demuxing exists to turn that batch back into **one renderable frame** for output. They solve different problems — and at 50 streams you usually drop the tiler entirely.

// 01

Threads in DeepStream

GStreamer is not single-threaded, and most confusion comes from assuming it is. Four kinds of threads run concurrently, each owning a different concern.

	1	Runs (the GLib main loop). Dispatches bus messages — this is where executes. Holds the kill switch. Does no frame processing.
	N sets	Each decodes one stream (demux, NVDEC, internal queues) on its own thread(s). Created/destroyed at runtime on add/remove.
	1+	Collects buffers from every source pad, forms the batch, pushes it through tiler → convert → osd → sink. The actual data plane.
	1	HTTP server (port 9000). Handles add/remove and *mutates the live pipeline* from this thread, concurrently with everything else.

▸ Why the boundaries matter

- **Errors cross a boundary via the bus.** A source fails on *its* thread, posts an error onto the bus (a thread-safe queue), and the main thread picks it up later in `bus_call`. That hand-off is why it's safe to call `loop.quit()` there — you never touch pipeline internals from the wrong thread.
- **The REST thread mutates while data flows.** This is the thread-safety hot spot, and the reason the new `nvstreammux` matters — it does the locking for dynamic pad add/release.
- **Where you block determines what stalls.** A `sync=1` sink blocks the downstream streaming thread, not the main thread. A pad probe runs on the streaming thread carrying the batch — block it and all frames freeze. Heavy work in `bus_call` stalls bus dispatch and GLib timers.
- **One stuck source doesn't freeze the others.** Sources decode independently and only rendezvous at the mux — which is exactly what makes per-stream remove-and-readd viable.

> RULE OF THUMB
>
> Data flows *sideways* through the streaming threads. Notifications flow *up* to the main thread via the bus. Commands come *in from the side* on the REST thread. Keep these three jobs on their own threads and never do long work on the main thread or inside a probe.

// 02

The bus mental model

Elements deep inside the pipeline can't call your Python directly — wrong thread, no reference to your code. So they **post messages onto the bus**, a thread-safe queue you can picture as the pipeline's mailbox. The main loop watches that mailbox and hands each message to one callback on the main thread.

bus_call.py

```python
def bus_call(bus, message, loop):
    t = message.type
    if t == Gst.MessageType.EOS:
        # Long-running server: a finite source ending must NOT tear
        # down the pipeline, or you can never add streams afterwards.
        print("[bus] EOS (a source ended) — server still running.")
    elif t == Gst.MessageType.WARNING:
        err, debug = message.parse_warning()
        print(f"[bus] WARNING: {err}: {debug}")
    elif t == Gst.MessageType.ERROR:
        err, debug = message.parse_error()
        print(f"[bus] ERROR: {err}: {debug}")
        # loop.quit()   <-- deliberately NOT called
    return True             # True = stay subscribed for the next message
```

Three things make this click:

- **It's a dispatcher, not a worker.** The bus delivers *every* message type — state changes, buffering, latency, QoS. This function inspects only three; everything else falls through and is ignored. That's normal.
- **`return True` means "stay subscribed."** Returning `False` removes the watch and you stop hearing from the pipeline entirely.
- **The three branches encode the server's personality.** A normal one-shot program quits on EOS and ERROR. This one refuses both, because it must outlive any single source. The `loop` argument is the kill switch passed in on purpose — and the design choice is to *not* pull it.

> THE TRADEOFF
>
> Silencing ERROR keeps the server up through transient per-source failures — but a genuinely fatal pipeline error is then only logged, never acted on. You get no automatic exit, so you must rely on the monitoring layer (section 07) to notice and react.

// 03

nvmultiurisrcbin & the new nvstreammux

▸ nvmultiurisrcbin

This bin packages three things that you would otherwise wire up by hand: one `nvurisrcbin` per stream (URI parsing + hardware decode), the new `nvstreammux` (batching), and an embedded REST control server. It exposes a single static src pad, so you link it directly downstream and never deal with dynamic request-pad linking yourself. That bundling is what makes runtime add/remove a one-line REST call instead of fragile pad surgery.

▸ Why the new nvstreammux

Set with `USE_NEW_NVSTREAMMUX=yes` *before* GStreamer loads. The legacy mux wanted a fixed batch size and explicit input dimensions up front and was brittle about sources appearing and disappearing. The new mux is built for a dynamic, unknown number of sources: it batches adaptively, tolerates sources joining and leaving mid-run, and does the locking required for safe dynamic add/remove.

A **batch**	An holding N independent frames side by side in GPU memory, plus . Travels as one . A scheduling/throughput construct — not a viewable image.
Tiling	Composite the N frames into one surface (the monitor-wall view). One sink, all streams visible.
Demuxing	splits the batch back into N single-stream branches — one sink/recording per camera.

> ADD / REMOVE OVER REST
>
> You target a single stream by its `camera_id`. Removing one frees its batch slot and releases its pad without disturbing the others; re-adding builds a fresh `nvurisrcbin`. Always let the remove complete before re-adding the same id, or you risk an id conflict or a momentary batch-cap rejection.

// 04

Scaling to 50 streams

The demo is hard-capped at 4 (`MAX_BATCH_SIZE = 4`, tiler `2×2`). Growing to 50 is mostly a matter of sizing the batch, fixing the output strategy, and respecting hardware limits.

▸ 1. Raise the batch ceiling

pipeline.py — source setup

```python
MAX_BATCH_SIZE = 50

src = make("nvmultiurisrcbin", "multiuri-src")
src.set_property("port", 9000)
src.set_property("ip-address", "0.0.0.0")
src.set_property("max-batch-size", MAX_BATCH_SIZE)
src.set_property("width", 1280)
src.set_property("height", 720)
# RTSP resilience (applied to every source the bin creates):
src.set_property("rtsp-reconnect-interval", 10)   # seconds; 0 disables
src.set_property("rtsp-reconnect-attempts", -1)   # -1 = retry forever
src.set_property("select-rtp-protocol", 4)        # 4 = TCP (lossy nets)
src.set_property("latency", 200)                  # jitter buffer, ms
src.set_property("drop-frame-interval", 0)        # 1+ = decode every Nth
```

▸ 2. Stop tiling — go headless

A 50-cell mosaic is useless to a human and wastes GPU on compositing. For analytics, drop the tiler and OSD and pull metadata off the batch with a probe, fanning results out to a message broker (Kafka via `nvmsgconv` + `nvmsgbroker`). Keep an *optional* tiled preview of a handful of streams on a separate demuxed branch if operators need eyes on it.

```
nvmultiurisrcbin → nvinfer (PGIE) → nvtracker → probe → nvmsgconv → nvmsgbroker
                                                  └─ extract NvDsBatchMeta, no rendering
```

▸ 3. Tune the new mux for many sources

config_mux.txt

```ini
[property]
algorithm-type=1
batch-size=50
adaptive-batching=1
# Cap aggregate throughput so a burst of sources can't overrun the GPU:
max-fps-control=1
overall-max-fps-n=900      # e.g. 50 streams × ~18 fps target
overall-max-fps-d=1
overall-min-fps-n=50
overall-min-fps-d=1
max-same-source-frames=1
```

> HARDWARE REALITY CHECK
>
> 50×1080p30 is heavy. The **NVDEC** decoder count is finite per GPU — exceed it and decode stalls. Mitigate by lowering input resolution, using `drop-frame-interval` to decode fewer frames, running inference every Nth frame (`interval` in the nvinfer config), watching GPU memory for the batched surface pools (`nvbuf-memory-type`, pool sizes), and scaling horizontally across multiple GPUs / processes when one card saturates.

// 05

RTSP failure & reconnect

Real cameras drop. Networks blip. Servers restart. The pipeline must treat a dead stream as routine, not exceptional. There are two layers of defense.

▸ Layer 1 — built-in reconnection

`nvurisrcbin` can recover transient RTSP drops by itself. With `rtsp-reconnect-interval` set, a dropped connection is retried automatically — no manual intervention. Prefer **TCP** (`select-rtp-protocol=4`) on lossy links to avoid UDP packet loss, and size the `latency` jitter buffer to your network. For most camera outages this is all you need.

PROPERTY	PURPOSE
`rtsp-reconnect-interval`	Seconds between auto-reconnect attempts (0 = off)
`rtsp-reconnect-attempts`	Max attempts; `-1` = retry forever
`select-rtp-protocol`	`0`=UDP, `4`=TCP — use TCP for reliability
`latency`	Jitter buffer in ms; raise on jittery networks
`udp-buffer-size`	Larger buffer absorbs bursts on UDP

▸ Layer 2 — application watchdog

Built-in reconnection handles connection drops, but not every failure mode: a camera that streams corrupt frames, a source wedged in an error state, a decoder that silently stops producing buffers. For those, you need an external watchdog that detects "no frames for X seconds" and forcibly removes-and-readds the stream over REST. That is section 08.

> FILE SOURCES
>
> For file inputs, `file-loop=True` already prevents the most common "stream ended" case by looping instead of emitting EOS. Manual remove/add is mainly for hard decode errors.

// 06

Keeping it alive

Persistence is a set of deliberate refusals to die, layered together:

1. **Never quit on EOS.** A finite source ending is normal; log and continue (section 02).
2. **Don't auto-quit on per-source ERROR.** Isolate the failure to one stream and let the watchdog recover it, rather than tearing down 49 healthy streams for one bad camera.
3. **Isolate sources structurally.** Because each source has its own threads and only meets the others at the mux, one failure stays local. Lean into that.
4. **Recover, don't restart.** Use REST remove/add to rebuild one source in place; never set the whole pipeline to `NULL` for a single-camera problem.
5. **Distinguish transient vs fatal.** Transient (drop, blip) → let built-in reconnect handle it. Persistent (repeated failures) → watchdog with backoff, and eventually mark the stream dead and alert.
6. **Wrap the process.** Run under a supervisor (systemd / Docker `--restart unless-stopped` / k8s) so even a true crash comes back automatically.

> ANTI-PATTERN
>
> Calling `loop.quit()` in the ERROR branch on a 50-camera server. One flaky camera should never be able to take down the whole fleet. Recover the single stream instead.

// 07

Monitoring & status

You can't recover what you can't see. The cheapest, most reliable health signal is "is each source still producing frames?" — captured with a single buffer probe on the mux's source pad, which fires once per batch and lets you record a last-seen timestamp per source.

monitor.py — per-source liveness probe

```python
import time, pyds
from collections import defaultdict

last_seen = {}                 # source_id -> monotonic timestamp
frame_count = defaultdict(int) # source_id -> frames in window

def mux_src_probe(pad, info, _):
    gst_buffer = info.get_buffer()
    if not gst_buffer:
        return Gst.PadProbeReturn.OK
    batch = pyds.gst_buffer_get_nvds_batch_meta(hash(gst_buffer))
    l = batch.frame_meta_list
    now = time.monotonic()
    while l is not None:
        fm = pyds.NvDsFrameMeta.cast(l.data)
        sid = fm.source_id
        last_seen[sid] = now
        frame_count[sid] += 1
        l = l.next
    return Gst.PadProbeReturn.OK   # keep this fast — it runs on the streaming thread

# attach once, on the bin's static src pad
src.get_static_pad("src").add_probe(Gst.PadProbeType.BUFFER, mux_src_probe, None)
```

From `last_seen` and `frame_count` you derive everything an operator needs:

- **Per-stream FPS** — reset `frame_count` on a GLib interval and divide by the window.
- **Liveness** — `now - last_seen[sid]` > threshold means the source has stalled even if it never posted an error.
- **GPU / decoder load** — poll NVML (or shell out to `nvidia-smi`) for utilization, memory, and decoder occupancy.
- **Bus events** — log every WARNING/ERROR with the offending stream id and a reconnect counter.
- **Expose it** — a small `/status` HTTP endpoint or a Prometheus exporter turns the dict above into dashboards and alerts.

> KEEP THE PROBE TRIVIAL
>
> The probe runs on the downstream streaming thread. Only read meta and stamp a dict — no I/O, no locks held long, no REST calls. Do the deciding and acting on the main thread or a dedicated watchdog thread.

// 08

Graceful reconnection

The watchdog ties monitoring to action: a periodic check reads the liveness dict, and any source that has gone silent past its grace period is rebuilt in place via REST — remove, wait for teardown, then add — with exponential backoff so a permanently-dead camera doesn't hammer the system.

watchdog.py

```python
import time, requests
from gi.repository import GLib

REST = "http://localhost:9000/api/v1/stream"
STALL_SECS   = 15           # no frames for this long => stalled
streams      = {}           # camera_id -> {"url":..., "source_id":..., "fails":0}
recovering   = set()

def post(action, cam_id, url, change):
    payload = {"key": "sensor", "value": {
        "camera_id": cam_id, "camera_name": cam_id,
        "camera_url": url, "change": change}}
    return requests.post(f"{REST}/{action}", json=payload, timeout=5)

def recover(cam_id):
    info = streams[cam_id]
    backoff = min(60, 2 ** info["fails"])      # 1,2,4,... capped at 60s
    info["fails"] += 1
    recovering.add(cam_id)
    try:
        post("remove", cam_id, info["url"], "camera_remove")
        time.sleep(backoff)                    # let teardown finish before re-add
        post("add", cam_id, info["url"], "camera_add")
        print(f"[watchdog] re-added {cam_id} (attempt {info['fails']}, backoff {backoff}s)")
    except Exception as e:
        print(f"[watchdog] recover {cam_id} failed: {e}")
    finally:
        recovering.discard(cam_id)

def check_health():
    now = time.monotonic()
    for cam_id, info in streams.items():
        sid = info["source_id"]
        if cam_id in recovering:
            continue
        stalled = (sid not in last_seen) or (now - last_seen[sid] > STALL_SECS)
        if stalled:
            if info["fails"] >= 8:             # give up, alert, stop retrying
                print(f"[watchdog] {cam_id} DEAD after {info['fails']} attempts")
                continue
            # run the blocking recover off the main loop
            GLib.Thread.new(f"recover-{cam_id}", lambda c=cam_id: recover(c))
        else:
            info["fails"] = 0                  # healthy again -> reset backoff
    return True                                # keep the timer alive

# check every 5s on the GLib main loop
GLib.timeout_add_seconds(5, check_health)
```

- **Detect** with the liveness dict from section 07 — catches silent stalls that never post an error.
- **Sequence** the remove before the add, with a wait, so the slot and pad are released before reuse.
- **Back off** exponentially; reset on recovery so a flapping camera self-heals without a storm of REST calls.
- **Give up gracefully** after a cap — mark the stream dead, alert, and stop retrying so a decommissioned camera doesn't churn forever.
- **Don't block the main loop** — run the blocking remove/sleep/add on a worker thread; let built-in RTSP reconnection (section 05) handle the easy cases so the watchdog only fires on genuine stalls.

> DIVISION OF LABOUR
>
> Built-in `rtsp-reconnect-interval` handles connection drops cheaply and in-place. The watchdog is the heavier hammer for hard failures the source can't recover from on its own. Layer both; don't make the watchdog do work the source bin already does.

// 09

Detector, license-plate recognition, tracking & ReID

Inference slots in between the source bin and the output. The canonical license-plate pipeline chains a primary detector, a tracker, and two secondary inference stages, all operating on the same batched buffer.

```
nvmultiurisrcbin → nvinfer PGIE → nvtracker → nvinfer SGIE-LPD → nvinfer SGIE-LPR → probe / osd / sink
                   detect vehicles      track + ReID      detect plate          read characters
```

▸ Primary detector (PGIE)

The first `nvinfer` runs once over the whole batch and attaches object metadata. For traffic, NVIDIA's **TrafficCamNet** (TAO) detects vehicles; swap in any model with its own config. Use `interval` to run detection every Nth frame at scale, and `batch-size` matched to the mux.

config_pgie.txt

```ini
[property]
gpu-id=0
net-scale-factor=0.00392156862745098
tlt-encoded-model=trafficcamnet.etlt
tlt-model-key=tlt_encode
labelfile-path=labels_trafficcamnet.txt
batch-size=50
network-mode=2            ; 0=FP32 1=INT8 2=FP16
num-detected-classes=4
interval=2               ; infer every 3rd frame to save compute
gie-unique-id=1
process-mode=1           ; 1 = primary
```

▸ Tracking

`nvtracker` assigns a persistent `object_id` to each detection across frames, so you read a plate once per vehicle instead of per frame. The low-level library is shared; a YAML selects the algorithm.

tracker setup

```python
tracker = make("nvtracker", "tracker")
tracker.set_property("tracker-width", 640)
tracker.set_property("tracker-height", 384)
tracker.set_property("ll-lib-file",
    "/opt/nvidia/deepstream/deepstream/lib/libnvds_nvmultiobjecttracker.so")
# Algorithm chosen by the config:
#   config_tracker_NvSORT.yml      -> fast, no appearance model
#   config_tracker_NvDCF_perf.yml  -> correlation filter, balanced
#   config_tracker_NvDeepSORT.yml  -> appearance ReID, most robust
tracker.set_property("ll-config-file", "config_tracker_NvDeepSORT.yml")
```

▸ ReID — identity that survives occlusion

Plain motion trackers lose an object when it's occluded or leaves and re-enters the frame, handing it a new id. **ReID** attaches an appearance embedding to each object so the tracker can re-associate the same identity later — and, because the embedding is comparable across cameras, it's the basis for multi-camera tracking. ReID is enabled inside the tracker config (NvDeepSORT or NvDCF-accuracy), pointing at a ReID feature-extraction model.

config_tracker_NvDeepSORT.yml (excerpt)

```yaml
ReID:
  reidType: 2                      # 2 = ReID-based association
  batchSize: 100
  reidFeatureSize: 256
  reidHistorySize: 100
  inferDims: [3, 256, 128]
  networkMode: 1                   # INT8 for throughput
  onnxFile: "resnet50_reid.onnx"   # appearance model
  modelEngineFile: "resnet50_reid.onnx_b100_gpu0_int8.engine"
  keepAspc: 1
  outputReidTensor: 1              # emit embeddings for cross-camera matching
```

▸ License-plate detection & recognition (SGIEs)

Two secondary `nvinfer` stages run only on the crops produced upstream. **LPDNet** finds the plate region inside each tracked vehicle; **LPRNet** reads the characters. Each is wired to operate on the previous stage's output via `operate-on-gie-id` and `operate-on-class-ids`, in secondary mode.

config_sgie_lpd.txt & config_sgie_lpr.txt (key fields)

```ini
; --- SGIE-1: LPDNet (plate detection on vehicle crops) ---
[property]
process-mode=2            ; 2 = secondary
gie-unique-id=2
operate-on-gie-id=1       ; consume PGIE (TrafficCamNet) output
operate-on-class-ids=0    ; e.g. the "car" class
tlt-encoded-model=lpdnet.etlt
network-type=0            ; detector

; --- SGIE-2: LPRNet (character recognition on plate crops) ---
[property]
process-mode=2
gie-unique-id=3
operate-on-gie-id=2       ; consume LPDNet output
tlt-encoded-model=lprnet.etlt
network-type=1            ; classifier / sequence
parse-classifier-func-name=NvDsInferParseCustomNVPlate   ; custom parser
; LPRNet also needs its character dictionary (dict.txt)
```

The recognized plate string lands as classifier metadata on each object; read it in the same probe you use for monitoring (walk `obj_meta.classifier_meta_list`) and forward it — with the stable tracker `object_id` and ReID embedding — to your message broker or database.

> REFERENCE IMPLEMENTATION
>
> NVIDIA ships a working version of exactly this chain (TrafficCamNet → LPDNet → LPRNet) in the open-source `deepstream_lpr_app`, with the matching config files and the LPRNet character dictionary. Start from those configs rather than writing them blank, and pull the models from TAO / NGC.

> COST AT SCALE
>
> Every added inference stage multiplies GPU load across all 50 streams. Use `interval` on the PGIE, let the tracker carry identity between detections (so SGIEs run on far fewer crops), keep ReID at INT8, and measure end-to-end latency with the monitoring probe before adding more stages.

// 10

Production checklist

AREA	DO THIS
Batching	`max-batch-size` = stream count; new mux with adaptive batching; FPS caps in the mux config.

Output	Headless + probe + message broker at scale; tile only a small operator subset on a demuxed branch.
RTSP	TCP transport, `rtsp-reconnect-interval` on, sized `latency` buffer.
Liveness	Per-source last-seen probe; FPS window; NVML for GPU/decoder load.
Recovery	Watchdog with sequenced remove/add, exponential backoff, dead-stream cap + alert.
Bus policy	Never quit on EOS; isolate per-source ERROR; act via monitoring, not `loop.quit()`.
Hardware	Respect NVDEC limits; watch GPU memory pools; scale across GPUs/processes when saturated.
Inference	PGIE `interval`, tracker-carried identity, INT8 ReID; benchmark latency before adding stages.
Process	Run under systemd / Docker restart policy / k8s so a true crash self-recovers.
Observability	`/status` endpoint or Prometheus; structured logs keyed by `camera_id`.

> THE ONE-LINE PHILOSOPHY
>
> A reliable DeepStream service is one that treats every individual stream as *expendable and replaceable* while treating the *pipeline* as immortal. Sources come and go; the batch adapts; the server never dies.

// end of guide · adapt configs to your DeepStream version · verify property names with `gst-inspect-1.0`